Higher Technological Institute
FACULITY OF ENGINEERING
ELECTRICALCOMMUNICATIONS DEPARTMENT

Smart Fan Controller

Automatic Temperature-Controlled DC Fan with Motor Protection

Computer Microprocessor

EEC 214

Submitted By:

Alaa Mohamed Abdo	**20243301**	**Group 3**
Mohamed Ahmed Salah	**20243150**	**Group 3**
Areeg Moustafa Mohamed	**20223115**	**Group 3**
Asmaa Khalid Moustafa	**20220693**	**Group 3**

Submitted To:

Prof.Dr. Mohamed Torad

2026

Abstract

This report presents the design and implementation of a Smart Fan Controller based on the ATMEGA32 microcontroller. The system automatically adjusts the speed of a DC fan according to ambient temperature readings obtained from a DHT11 sensor. Three operating speed levels are implemented using Pulse Width Modulation (PWM): the fan remains off below 25°C, operates at medium speed (70% duty cycle) between 25°C and 35°C, and runs at full speed above 35°C. In addition to the automatic mode, the system supports manual override via a Bluetooth module (HC-05), allowing the user to select from four speed levels (OFF, LOW, MEDIUM, HIGH) through a mobile terminal application. Real-time temperature, humidity, and operating mode are displayed on an I2C 16×2 LCD. A motor protection feature is also incorporated: an ACS712 current sensor monitors the motor current and automatically cuts power if an overcurrent condition is detected, thereby preventing motor burnout [1][2][3].

Acknowledgement

First, thanks to ALLAH, the most merciful, the most gracious, for this moment has come and this work has been accomplished. Thanks to the Higher Technological Institute of 10th Ramadan for preparing me to be a successful engineer and lifting me up to achieve this training in an environment that's full of encouragement and motivation.

My deepest thanks to my parents, whose sacrifices, endless support, and belief in my abilities have been the foundation of my educational journey. Their prayers and continuous encouragement have always been my source of strength and motivation.

My highest gratitude to the faculty members of the Department of Electrical & Computers Engineering for all the support they provide us during the period of study at the Institute. Special thanks to instructor **Prof.Dr. Mohammed Torad** for her help and knowledge in the field of microcontroller. His professional touches are sensed within every phase of this Computer Microprocessor course.

Table of Contents

Table of Figures

Project Title

Smart Fan Controller

Automatic Temperature-Controlled DC Fan with Overcurrent Motor Protection

Project Description

The Smart Fan Controller is an embedded system designed to provide intelligent, energy-efficient control of a DC cooling fan based on real-time ambient temperature measurements. The system operates in two primary modes: Automatic Mode and Manual Mode.

Automatic Mode

In Automatic Mode, the ATMEGA32 microcontroller continuously polls the DHT11 temperature and humidity sensor. Depending on the temperature reading, the PWM duty cycle driving the DC fan is adjusted as follows [1]:

- Temperature < 25°C → Fan OFF (OCR1A = 0)
- 25°C ≤ Temperature < 35°C → Medium Speed (OCR1A = 180, ~70% duty cycle)
- Temperature ≥ 35°C → High Speed (OCR1A = 255, 100% duty cycle)

Manual Mode

Manual Mode is activated via Bluetooth commands received from the HC-05 module. The user sends a character command through a mobile terminal application, and the system responds accordingly [2]:

- Command '0' → Fan OFF
- Command '1' → Low Speed (OCR1A = 120)
- Command '2' → Medium Speed (OCR1A = 180)
- Command '3' → High Speed (OCR1A = 255)
- Command 'A' → Return to Automatic Mode

Motor Protection

The system includes an ACS712 current sensor connected to an ADC channel of the ATMEGA32. The sensor continuously monitors the current drawn by the DC motor. If the measured current exceeds a predefined threshold, the

microcontroller disables the motor driver to prevent damage due to overload or mechanical failure [3].

Display

An I2C 16×2 LCD displays the current temperature (in °C), the active mode character, and a status message on the second line (e.g., 'AUTO HIGH', 'LOW SPEED', 'FAN OFF'). This provides the user with clear, real-time feedback of the system state [4].

Aim of the Project

The primary aims and objectives of this project are as follows:

- To design and implement a real-time temperature-controlled fan system using the ATMEGA32 microcontroller.
- To utilize the PWM peripheral of the ATMEGA32 (Timer1, Fast PWM 8-bit mode) to achieve variable-speed fan control.
- To interface the DHT11 digital temperature and humidity sensor with the microcontroller using a custom driver [1].
- To provide wireless manual override capability via a Bluetooth module (HC-05) and a mobile terminal application using the UART peripheral [2].
- To display system status in real-time on an I2C 16×2 LCD using the TWI (I2C) hardware peripheral of the ATMEGA32 [4].
- To implement a motor overcurrent protection mechanism using the ACS712 current sensor and the built-in ADC of the ATMEGA32 [3].
- To gain practical experience with embedded C programming for AVR microcontrollers, peripheral configuration, and real-world sensor integration.

Project Block Diagram

The circuit diagram below shows the pin-level connections between all hardware components and the ATMEGA32 microcontroller.

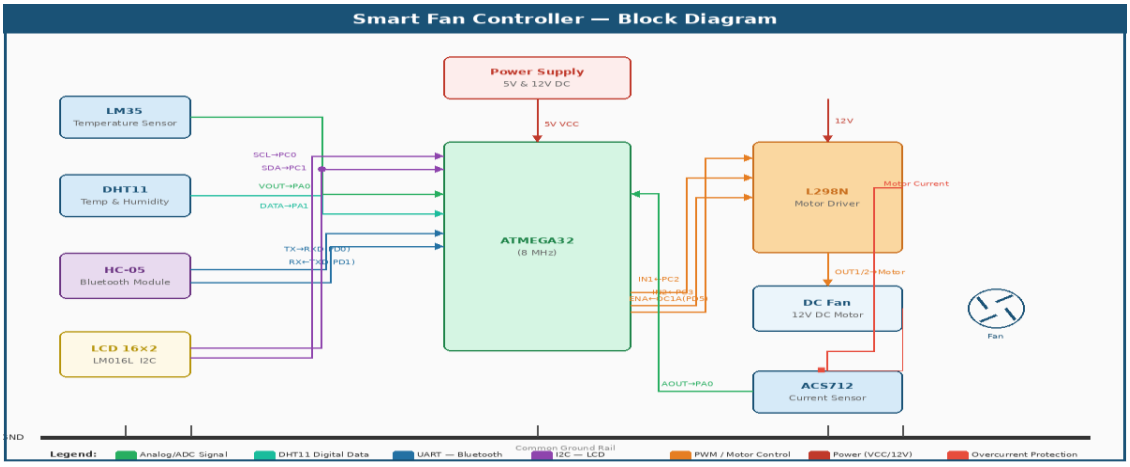

Figure1: Smart Fan Controller Block Diagram

This figure illustrates the high-level system architecture, centered around the ATmega32 microcontroller. It depicts the integration of the DHT11 and LM35 sensors for environmental monitoring, the HC-05 module for wireless UART communication, and the L298N driver for PWM-based motor control. The diagram also highlights the ACS712 current sensor integrated into the feedback loop for motor overcurrent protection.

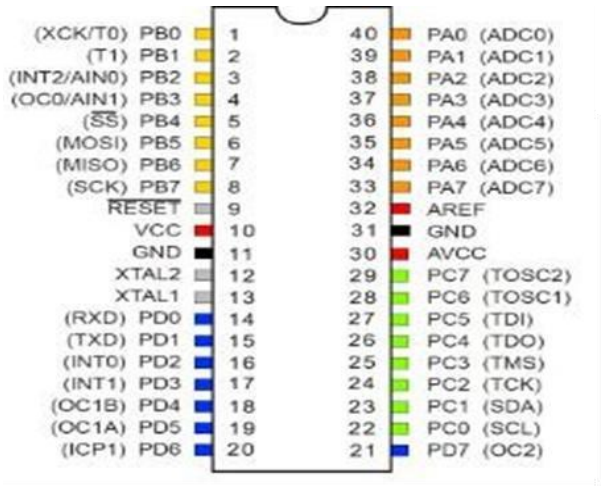

Figure2: ATmega32

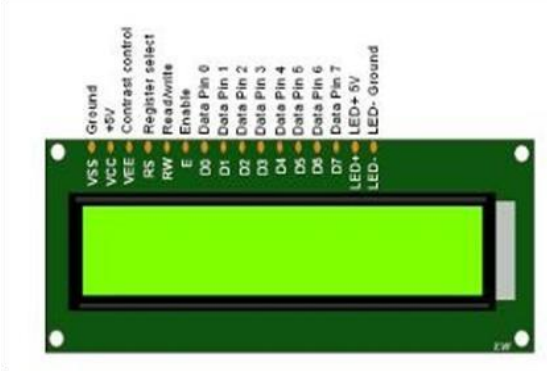

Figure3: LCD 16x2

Show as Figure 2 ATmega32 Microcontroller The central processing unit utilized for real-time data execution and peripheral management.

Show as Figure 3 16x2 LCD Interface used to display real-time environmental data and the status of the traffic control system.

Project Circuit Diagram

The circuit diagram below shows the pin-level connections between all hardware components and the ATMEGA32 microcontroller.

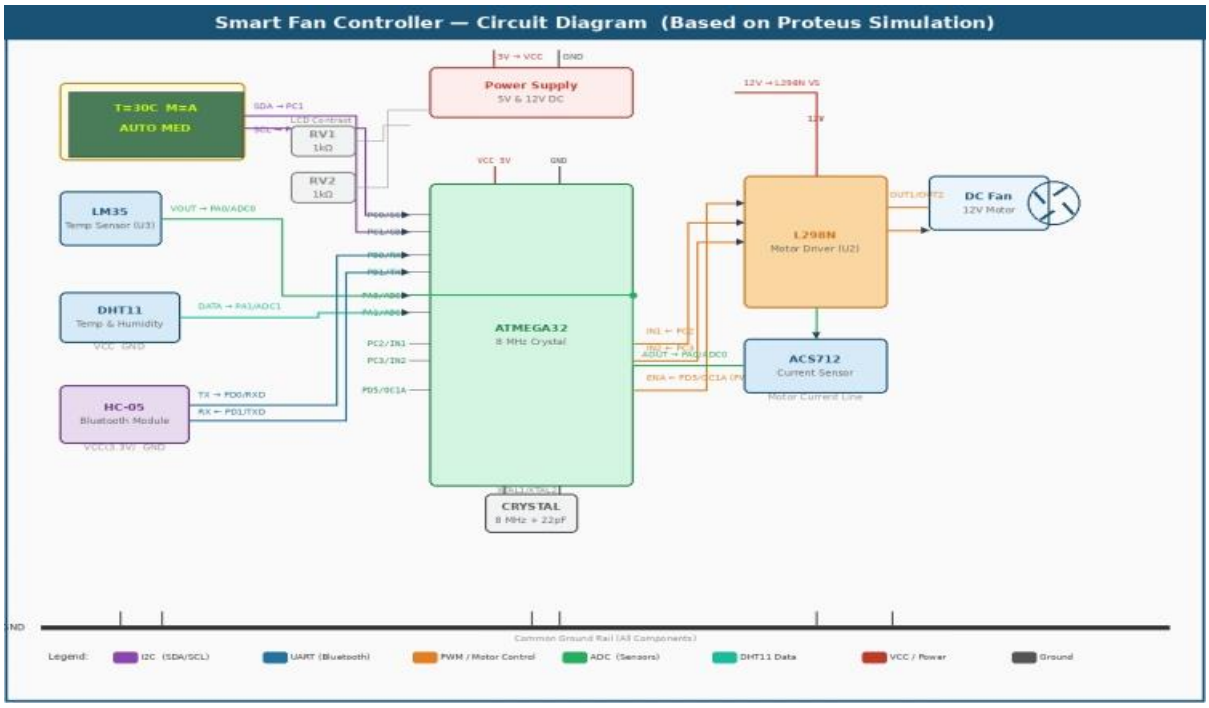

Figure 4: Smart Fan Controller Circuit Diagram

Shown above is the detailed wiring schematic developed for the system. It specifies the pin mapping for the ATmega32, including the I2C interface (SDA/SCL) for the LCD, the ADC channels for sensor data acquisition, and the OC1A (PD5) output used for high-resolution PWM speed regulation of the DC fan.

Project Construction

The project was constructed in several stages, from schematic design to final hardware assembly and software testing [1][2][3][4].

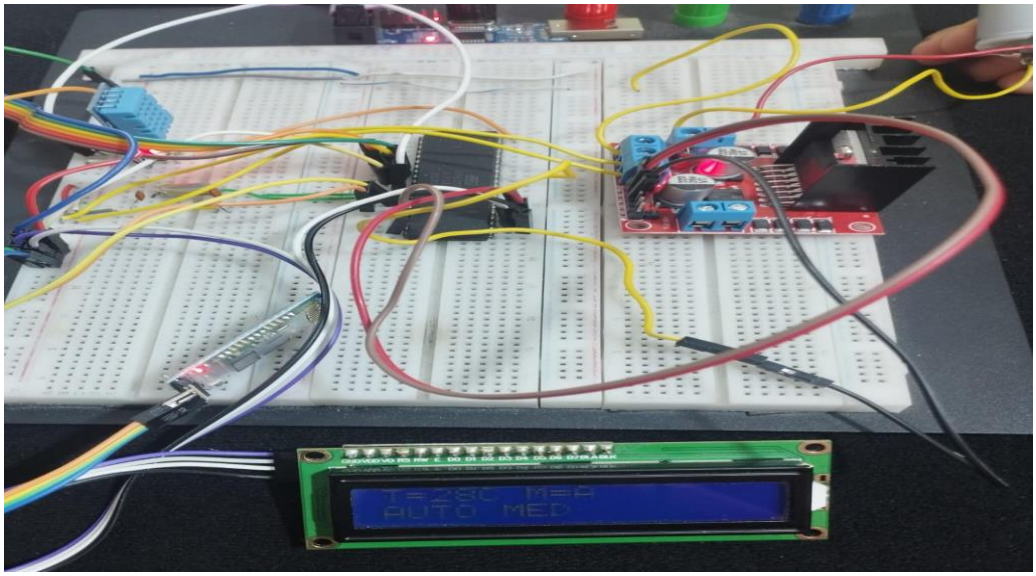

Figure5: The Project Constructed on a breadboard

Figure 5 displays the final hardware implementation on a breadboard. The setup validates the real-time interaction between the embedded C code and the physical components, showing the LCD active with live telemetry (28°C) and the L298N module successfully driving the cooling fan under regulated power conditions.

Simulation software:

The following screenshot shows the actual Proteus ISIS simulation of the circuit, confirming the hardware connections and verifying system behavior at 30°C ambient temperature (displaying AUTO MED mode on the LCD) [1][2].

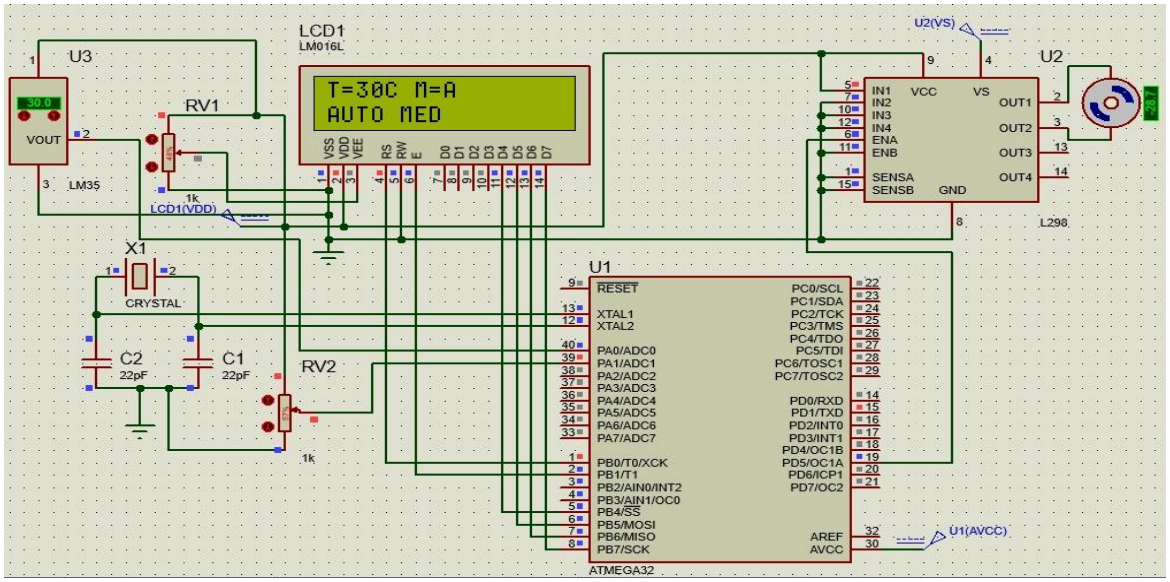

Figure 6: Proteus ISIS Simulation Smart Fan Controller

Show as Figure 6 demonstrates the functional verification of the system logic within the Proteus environment. The simulation confirms that the firmware correctly interprets a temperature of 30°C to trigger the "AUTO MED" speed mode, while successfully updating the 16x2 LCD interface via the virtual I2C bus.

Project Operation -Program Description Language (PDL)

The following PDL describes the logical flow of the main program [1][2][3][4]:

```
BEGIN Smart Fan Controller

INITIALIZE I2C, LCD, PWM (Timer1 Fast PWM 8-bit), UART (9600 baud)

  INITIALIZE Motor Direction: IN1=HIGH, IN2=LOW

  SET mode = 'A'  // Default: Automatic mode

  LOOP FOREVER:

    // — Bluetooth Input —

    IF UART data available THEN

      mode = received character

    END IF
```

```
// — Read Sensor —

IF DHT11 read SUCCESS THEN

  Display temperature and mode on LCD Line 1

  // — Manual Mode —

  IF mode = '0' THEN  Set PWM=0,   Display 'FAN OFF'

  IF mode = '1' THEN  Set PWM=120, Display 'LOW SPEED'

  IF mode = '2' THEN  Set PWM=180, Display 'MED SPEED'

  IF mode = '3' THEN  Set PWM=255, Display 'HIGH SPEED'



  // — Automatic Mode —

  IF mode = 'A' THEN

    IF temp < 25   THEN  Set PWM=0,   Display 'AUTO OFF'

    IF temp < 35   THEN  Set PWM=180, Display 'AUTO MED'

    ELSE                 Set PWM=255, Display 'AUTO HIGH'

  END IF

ELSE
```

```
Display 'Sensor Error' on LCD

  END IF



  // — Motor Protection –READ ADC from ACS712



  IF measured current > threshold THEN

    Set PWM = 0

// Cut motor power

    Display 'MOTOR FAULT'

  END IF

  DELAY 1000 ms

END LOOP

END Smart Fan Controller
```

Project Program Listing

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>

#include "i2c_lcd.h"
#include "DHT11.h"

char buffer[16];
char bluetooth;
char mode = 'A';

uint8_t temp;
uint8_t hum;

//================ I2C =================
void I2C_Init()
{
TWBR = 32;
TWSR = 0x00;
}

//================ PWM =================
void PWM_Init()
{
// OC1A = PD5
DDRD |= (1<<PD5);
```

```
// Fast PWM 8-bit
TCCR1A = (1<<COM1A1) | (1<<WGM10);

TCCR1B = (1<<WGM12) | (1<<CS11);

OCR1A = 0;
}

//================ UART =================
void UART_Init()
{
unsigned int ubrr = 51;

UBRRH = (ubrr >> 8);
UBRRL = ubrr;

UCSRB = (1<<TXEN) | (1<<RXEN);

UCSRC = (1<<URSEL) | (1<<UCSZ0) | (1<<UCSZ1);
}

char UART_Receive()
{
if(UCSRA & (1<<RXC))
{
        return UDR;
}

return 0;
}
```

```
//================ MAIN =================
int main(void)
{
//========== INIT ==========
I2C_Init();

LCD_Init();

PWM_Init();

UART_Init();

//========== MOTOR DRIVER ==========
// IN1 = PC2
// IN2 = PC3

DDRC |= (1<<PC2) | (1<<PC3);

// ????? ????
PORTC |= (1<<PC2);

PORTC &= ~(1<<PC3);

LCD_Clear();

while(1)
{
        //========== Bluetooth ==========
        bluetooth = UART_Receive();
```

```
if(bluetooth != 0)
{
	mode = bluetooth;
}

//========== DHT11 ==========
if(DHT11_ReadData(&temp, &hum))
{
	//========== FIRST LINE ==========
	LCD_SetCursor(0,0);

	LCD_String("T=");

	itoa(temp, buffer, 10);

	LCD_String(buffer);

	LCD_String("C ");

	LCD_String("M=");

	LCD_Char(mode);

	LCD_String("   ");

	//========== SECOND LINE ==========

	//========== MANUAL ==========
	if(mode == '0')
```

```
{
        OCR1A = 0;

        LCD_SetCursor(1,0);

        LCD_String("FAN OFF       ");
}

else if(mode == '1')
{
        OCR1A = 120;

        LCD_SetCursor(1,0);

        LCD_String("LOW SPEED     ");
}

else if(mode == '2')
{
        OCR1A = 180;

        LCD_SetCursor(1,0);

        LCD_String("MED SPEED     ");
}

else if(mode == '3')
{
        OCR1A = 255;
```

```
		LCD_SetCursor(1,0);

		LCD_String("HIGH SPEED     ");
}

//========== AUTO ==========
else if(mode == 'A')
{
		if(temp < 25)
		{
			OCR1A = 0;

			LCD_SetCursor(1,0);

			LCD_String("AUTO OFF       ");
		}

		else if(temp < 35)
		{
			OCR1A = 180;

			LCD_SetCursor(1,0);

			LCD_String("AUTO MED       ");
		}

		else
		{
			OCR1A = 255;
```

```
                    LCD_SetCursor(1,0);

                    LCD_String("AUTO HIGH     ");
                }
            }
        }

        else
        {
            LCD_SetCursor(0,0);

            LCD_String("Sensor Error   ");
        }

        _delay_ms(1000);
}
}
```

Description of the Program

The firmware is written in embedded C targeting the ATMEGA32 at 8 MHz using AVR-GCC. The program is structured into four initialization routines and a main polling loop.

I2C_Init()

Initializes the Two-Wire Interface (TWI/I2C) by setting the bit rate register TWBR = 32 and clearing the prescaler bits in TWSR. This produces an I2C clock frequency of approximately 100 kHz, suitable for the LCD module [4].

PWM_Init()

Configures Timer1 in Fast PWM 8-bit mode (WGM10 + WGM12) with the OC1A output on PD5 set to non-inverting mode (COM1A1). The timer uses a prescaler

of 8 (CS11), yielding a PWM frequency of approximately 3.9 kHz at 8 MHz. OCR1A controls the duty cycle: 0 = 0% (fan off), 120 ≈ 47%, 180 ≈ 70%, 255 = 100% [1].

UART_Init()

Initializes the USART peripheral for 9600 baud, 8 data bits, 1 stop bit, no parity. The baud rate register UBRRL = 51 is calculated from the formula: UBRR = (F_CPU / (16 × Baud)) − 1. Both transmitter and receiver are enabled [2].

UART_Receive()

A non-blocking receive function that checks the RXC flag in UCSRA. If a byte is available in the UDR register, it returns the character; otherwise it returns 0, allowing the main loop to continue without stalling.

Main Loop Logic

The main while(1) loop performs the following tasks on every iteration [1][2][3][4]:

- Checks for a Bluetooth command: if a non-zero character is received via UART, it updates the mode variable.
- Calls DHT11_ReadData() to obtain fresh temperature and humidity values.
- Updates the LCD first line with temperature in Celsius and the current mode character.
- Selects the appropriate OCR1A duty cycle based on the current mode (manual or automatic) and updates the second LCD line with the status string.
- Waits 1000 ms (1 second) before the next sensor read — respecting the DHT11 minimum sampling interval.

Motor Protection (Extension)

The ACS712 current sensor outputs an analog voltage proportional to the measured current. The ADC on PA0 is read periodically. The raw ADC value is compared against a threshold corresponding to the maximum rated current. If exceeded, OCR1A is set to 0 to stop the fan and a fault message is displayed [3].

References

1. Atmel Corporation. "ATmega32 Datasheet." Atmel, 2011. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/doc2503.pdf
2. Aosong Electronics. "DHT11 Humidity & Temperature Sensor Datasheet." Aosong, 2010.
3. Allegro MicroSystems. "ACS712 Fully Integrated, Hall Effect-Based Linear Current Sensor Datasheet." Allegro, 2006.
4. STMicroelectronics. "L298N Dual Full-Bridge Driver Datasheet." ST, 2000. Available: https://www.st.com/resource/en/datasheet/l298.pdf